MAD-1 Project Report – Placement Portal Application

Student Details

Name: Mudireddy Krishna Vaishnavi
Roll Number: 24F2009315
Email: 24f2009315@ds.study.iitm.ac.in

About Me:
I am a Computer Science student interested in backend development, database-driven applications, and web application design using Python and Flask.

Project Details

Project Title: Placement Portal Application

Problem Statement:
This project develops a web-based Placement Portal to support campus recruitment activities. The system allows students to explore placement drives and apply for them, companies to create and manage drives, and administrators to monitor the overall placement process.

Approach:
The system was designed by identifying the core entities involved in placement management: users, students, companies, placement drives, and applications. A relational database schema models these entities and their relationships. Flask handles routing and backend logic, while Jinja2 templates are used to render dynamic pages. SQLite was used as the database because it is lightweight and integrates easily with Flask.

AI / LLM Declaration

ChatGPT (OpenAI) was used mainly for debugging assistance, improving route logic structure, generating documentation, and preparing the API YAML specification.

- Backend (Flask + DB): Allowed 35%, Used 20%
- Frontend (HTML): Allowed 28%, Used 18%
- Styling / UI: Allowed 8%, Used 6%
- CRUD Logic: Allowed 20%, Used 15%
- Extras (API / Docs): Allowed 9%, Used 7%

All main implementation, integration, and testing were completed manually.

Technologies Used

- Python: Core programming language
- Flask: Backend web framework
- Flask-SQLAlchemy: Database ORM
- Flask-Login: Authentication and session handling
- SQLite: Relational database
- HTML / CSS / Bootstrap: Frontend interface
- Jinja2: Dynamic template rendering
- YAML / OpenAPI: API documentation

Database Schema

The system uses a relational database consisting of five main tables:

- Users: stores account information such as name, username, password, role, and account status.
- Students: stores student profile details including roll number, phone number, department, degree, batch year, CGPA, resume link, and placement status.
- Companies: stores company profile information including company name, HR contact details, website, and approval status.
- Placements: stores placement drive information such as company, title, description, eligibility criteria, application deadline, and drive status.
- Applications: stores the relationship between students and placement drives along with application date and status.

Relationships:

- One company can create multiple placement drives.
- One student can apply to multiple drives.
- One drive can receive multiple applications.
- Each student and company account is linked to a corresponding user record.

This design ensures normalized storage and efficient retrieval of placement data.

API Design

The project provides REST-style JSON APIs for core resources including students, companies, placement drives, and applications. These APIs support basic CRUD operations.

Example endpoints:

- `GET /api/companies` – retrieve all companies
- `POST /api/companies` – create a company

- `GET /api/students` – retrieve all students
- `POST /api/students` – create a student
- `GET /api/drives` – retrieve all placement drives
- `POST /api/drives` – create a placement drive
- `GET /api/applications` – retrieve all applications
- `POST /api/applications` – submit an application

The complete API specification is documented in the `api.yaml` file included with the project.

Architecture and Features

The application follows a modular Flask architecture. `app.py` initializes the application and registers blueprints, while the main logic is organized into separate modules for authentication, admin operations, company workflows, student workflows, drives, and applications. HTML pages are rendered using Jinja2 templates in the `templates` folder, and SQLite is used to store users, student profiles, company profiles, placement drives, and applications. REST-style APIs are also implemented and documented using `api.yaml`.

Implemented Features

- Student: register/login, view approved companies and drives, apply to drives, update profile, track application status.
- Company: register, create drives, view applicants, update application status, mark drives as completed.
- Admin: approve/reject companies and drives, manage students and companies, and view dashboard statistics.
- System: role-based access control, session authentication, SQLite integration, Jinja2-rendered pages, and YAML-documented REST APIs.

Video Demonstration

Video Link:

2026-03-09 17-20-47.mkv

https://drive.google.com/file/d/145oGVoo6vpYfxyPLk2avdnWeXbZYYMU-/view?usp=sharing